

Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа №1

За седьмой семестр

По дисциплине: «Модели решения задач в интеллектуальных системах»

Тема: «Реализация многослойного персептрона для решения задачи
«исключающее ИЛИ» »

Выполнил:

Студент 4 курса
Группы ИИ-26(1)
Прокопюк А.Д.

Проверил:

Никитин С.К.

Брест 2026

Цель работы: Реализовать многослойный персептрон (MLP) архитектуры 2-2-1 с сигмоидной функцией активации для решения задачи типа «исключающее ИЛИ» (XOR), в которой входные и выходные значения представлены не бинарными 0 и 1, а произвольными вещественными (целыми) числами из диапазона от -10 до 10 , соответствующими логическим классам 0 и 1. Провести сравнение эффективности обучения и качества решения задачи многослойным персептроном в сопоставлении с однослойным персептроном, показав принципиальную линейную неразделимость задачи XOR.

Задачи лабораторной работы

1. Реализовать многослойный персептрон архитектуры 2-2-1 (2 входных нейрона, 2 нейрона скрытого слоя, 1 выходной нейрон) с сигмоидной функцией активации на всех слоях.
2. Реализовать алгоритм обучения сети методом обратного распространения ошибки (backpropagation) в онлайн-режиме (обновление весов после каждого примера).
3. Обучить сеть на полной таблице истинности (4 примера) своего варианта до достижения заданного критерия останова по суммарной ошибке $E_s \leq E_e$.
4. Реализовать и обучить однослойный персептрон (без скрытого слоя, один сигмоидный нейрон) на тех же данных и зафиксировать результат его обучения.
5. Сравнить многослойный и однослойный персептроны по следующим критериям:

- количество эпох до сходимости (либо факт недостижения критерия останова за заданное максимальное число эпох),

- итоговая суммарная ошибка, точность (accuracy) воспроизведения всех 4 примеров таблицы истинности.

6. Реализовать режим функционирования обученной сети: пользователь вводит пару чисел (A, B) из диапазона $[-10; 10]$ (не обязательно совпадающих с c_0 и c_1), сеть выдаёт выходное значение $\hat{y}$ и указывает, к какому классу (c_0 или c_1) оно ближе.
7. Сформулировать вывод по результатам сравнения (см. вопросы ниже).

Вопросы для вывода

1. Смог ли однослойный перцептрон решить задачу XOR (с числовыми классами вместо 0/1)? Объясните полученный результат с точки зрения линейной делимости данных.
2. Смог ли многослойный перцептрон 2-2-1 корректно решить задачу? За какое число эпох была достигнута сходимость?
3. Как изменение диапазона выходных значений (произвольные числа вместо 0 и 1) повлияло на процесс обучения и выбор функции активации выходного слоя?
4. Почему для решения задачи XOR необходим хотя бы один скрытый слой?
5. Как результаты, полученные для числовых классов из диапазона $[-10; 10]$, соотносятся с классическим случаем XOR (0 и 1)? Изменилась ли сложность задачи?

Вариант 12 ($c_0 = -10$, $c_1 = 9$)

A	B	$A \oplus B$
-10	-10	-10
-10	9	9
9	-10	9
9	9	-10

Многослойный перцептрон конфигурации [2, 2, 1] состоит из входного слоя на два признака, скрытого слоя из двух нейронов для нелинейного разделения пространства XOR и выходного слоя с одним нейроном. В качестве активации на всех слоях используется сигмоида, обучение ведется методом обратного распространения ошибки с минимизацией среднеквадратичной ошибки (MSE), а качество контролируется по проценту верно классифицированных точек (Ассигасу) после денормализации выходов обратно в диапазон варианта.

Однослойный перцептрон конфигурации [2, 1] представляет собой линейный классификатор без скрытых слоев, состоящий только из входного слоя на два признака и одного выходного нейрона с сигмоидной активацией. Из-за отсутствия скрытых слоев он не способен выполнять нелинейное разделение пространства, что делает невозможным решение задачи XOR (его максимальная точность останавливается на уровне 50%, а ошибка фиксируется на плато). Обучение также ведется методом обратного

распространения ошибки с минимизацией MSE, но из-за линейной неделимости алгоритм приходит к неизбежному тупику.

Код программы:

```
import numpy as np
import matplotlib.pyplot as plt

def sigmoid(wsum):
    return 1 / (1 + np.exp(-wsum))

def der_sigmoid(y):
    return y * (1 - y)

def mse(yj, ej):
    return 0.5 * (yj - ej) ** 2

def normalize(val, min_val=-10, max_val=9):
    return (val - min_val) / (max_val - min_val)

def denormalize(val, min_val=-10, max_val=9):
    return min_val + val * (max_val - min_val)

def calculate_accuracy(net, X_norm, y_raw_true, c0=-10, c1=9):
    correct = 0
    threshold_mid = (c0 + c1) / 2

    for i in range(len(X_norm)):
        pred_norm = net.forward(X_norm[i:i+1])[0][0]
        pred_denorm = denormalize(pred_norm, min_val=c0, max_val=c1)

        assigned_class = c1 if pred_denorm >= threshold_mid else c0
        true_class = y_raw_true[i][0]

        if assigned_class == true_class:
            correct += 1

    return (correct / len(X_norm)) * 100

class NN:
    def __init__(self, layer_sizes):
        self.layer_sizes = layer_sizes
        self.num_layers = len(layer_sizes)

        self.weights = []
        self.thresholds = []

        for i in range(self.num_layers - 1):
            limit = 1 / np.sqrt(layer_sizes[i])
```

```

        w = np.random.uniform(-limit, limit, (layer_sizes[i], layer_sizes[i +
1]))

        T = np.zeros((1, layer_sizes[i + 1]))

        self.weights.append(w)
        self.thresholds.append(T)

    def forward(self, X):
        self.activations = [X]
        current_x = X    #y_j

        for i in range(self.num_layers-1):
            wsum = np.dot(current_x, self.weights[i]) - self.thresholds[i]
            current_x = sigmoid(wsum)
            self.activations.append(current_x)

        return current_x

    def backward(self, ethalon, lr):
        error = (self.activations[-1] - ethalon) * der_sigmoid(self.activations[-
1])

        for i in range(self.num_layers-2, -1, -1):
            inputs = self.activations[i]
            self.weights[i] -= np.dot(inputs.T, error) * lr
            self.thresholds[i] += np.sum(error, axis=0, keepdims=True) * lr

            if i > 0:
                error = np.dot(error, self.weights[i].T) *
der_sigmoid(self.activations[i])

    def train_network(layer_sizes, X, y, epochs=20000, lr=0.5, target_error=0.0001):
        net = NN(layer_sizes)
        error_history = []

        final_epoch = epochs
        for epoch in range(epochs):
            total_error = 0
            for i in range(len(X)):
                pred = net.forward(X[i:i+1])
                total_error += np.sum(mse(pred, y[i:i+1]))
                net.backward(y[i:i+1], lr)

            error_history.append(total_error)

            if epoch % 2000 == 0:
                acc = calculate_accuracy(net, X, y_raw, c0, c1)
                print(f"Epoch {epoch}, Error: {total_error:.5f}, Accuracy:
{acc:.1f}%")

            if total_error <= target_error:
                error_history.extend([total_error] * (epochs - len(error_history)))

```

```

        final_epoch = epoch + 1
        break

    return net, error_history, final_epoch, error_history[-1]

def predict_point(net, a, b, c0=-10, c1=9):
    raw_input = np.array([[a, b]])
    norm_input = normalize(raw_input, min_val=c0, max_val=c1)

    pred_norm = net.forward(norm_input)[0][0]

    pred_denorm = denormalize(pred_norm, min_val=c0, max_val=c1)

    threshold_mid = (c0 + c1) / 2
    assigned_class = c1 if pred_denorm >= threshold_mid else c0

    print(f"Input: A={a}, B={b} Output: {pred_denorm:.2f} Predicted class: {assigned_class}")

def plot_mse_history(history_mlp, history_perceptron):
    plt.figure(figsize=(10, 6))
    plt.plot(history_mlp, label='MLP', color='blue', linewidth=2)
    plt.plot(history_perceptron, label='SLP', color='red', linestyle='--',
linewidth=2)

    plt.title('SLP vs cooler SLP(MLP)')
    plt.xlabel('Epochs')
    plt.ylabel('MSE')
    plt.legend()
    plt.grid(True, which="both", ls="--")
    plt.show()

def plot_decision_boundary(net, c0=-10, c1=9):
    padding = 1.0
    x_min, x_max = c0 - padding, c1 + padding
    y_min, y_max = c0 - padding, c1 + padding

    xx, yy = np.meshgrid(np.linspace(x_min, x_max, 200), np.linspace(y_min,
y_max, 200))
    grid_coords = np.c_[xx.ravel(), yy.ravel()]

    grid_norm = normalize(grid_coords, min_val=c0, max_val=c1)
    preds = np.array([net.forward(pt.reshape(1, -1))[0][0] for pt in grid_norm])
    preds_denorm = denormalize(preds, min_val=c0, max_val=c1)
    ZZ = preds_denorm.reshape(xx.shape)

    threshold_mid = (c0 + c1) / 2

    plt.figure(figsize=(8, 6))

```

```

plt.contourf(xx, yy, ZZ, levels=[c0, threshold_mid, c1], colors=['lightblue',
'lightpink'], alpha=0.6)
plt.contour(xx, yy, ZZ, levels=[threshold_mid], colors='red', linewidths=2)

X_orig = np.array([[c0, c0], [c0, c1], [c1, c0], [c1, c1]])
y_orig = np.array([c0, c1, c1, c0])

for i, pt in enumerate(X_orig):
    dot_color = 'darkblue' if y_orig[i] == c0 else 'darkred'
    plt.scatter(pt[0], pt[1], color=dot_color, edgecolors='black', s=180,
linewidth=1.5, zorder=5)

plt.xlim(x_min, x_max)
plt.ylim(y_min, y_max)

plt.title('Perceptron Decision Boundary')
plt.xlabel('A')
plt.ylabel('B')
plt.grid(True, linestyle='--', alpha=0.3)
plt.xticks(np.arange(-11, 11, 1))
plt.yticks(np.arange(-11, 11, 1))
plt.show()

if __name__ == "__main__":
    c0, c1 = -10, 9
    X_raw = np.array([[c0, c0], [c0, c1], [c1, c0], [c1, c1]])
    y_raw = np.array([c0, c1, c1, c0])

    X = normalize(X_raw)
    y = normalize(y_raw)

    epochs = 200000
    lr = 1e-1

    print("training (2-2-1)...")
    net_mlp, history_mlp, final_epoch_mlp, final_error_mlp = train_network([2, 2,
1], X, y, epochs=epochs, lr=lr)
    final_acc_mlp = calculate_accuracy(net_mlp, X, y_raw, c0, c1)

    print("training (2-1)...")
    net_perceptron, history_perceptron, final_epoch_slp, final_error_slp =
train_network([2, 1], X, y, epochs=epochs, lr=lr)
    final_acc_slp = calculate_accuracy(net_perceptron, X, y_raw, c0, c1)

    print(f"MLP Epochs: {final_epoch_mlp}    Error: {final_error_mlp}    Accuracy:
{final_acc_mlp}")
    print(f"SLP Epochs: {final_epoch_slp}    Error: {final_error_slp}    Accuracy:
{final_acc_slp}")

    print("\n--- test case:")
    test_cases = [(-10, -10), (-10, 9), (9, -10), (9, 9), (0, 0), (5, -5)]

```

```

for a, b in test_cases:
    predict_point(net_mlp, a, b, c0, c1)

plot_mse_history(history_mlp, history_perceptron)
plot_decision_boundary(net_mlp)

while True:
    user_input = input("\nInput nubers A and B through the space(or 'q' for
exit): ").strip()
    if user_input.lower() in ['exit', 'выход', 'q']:
        print("Exit...")
        break

    try:
        parts = user_input.split()
        if len(parts) != 2:
            print("Error: you must input two numbers!")
            continue

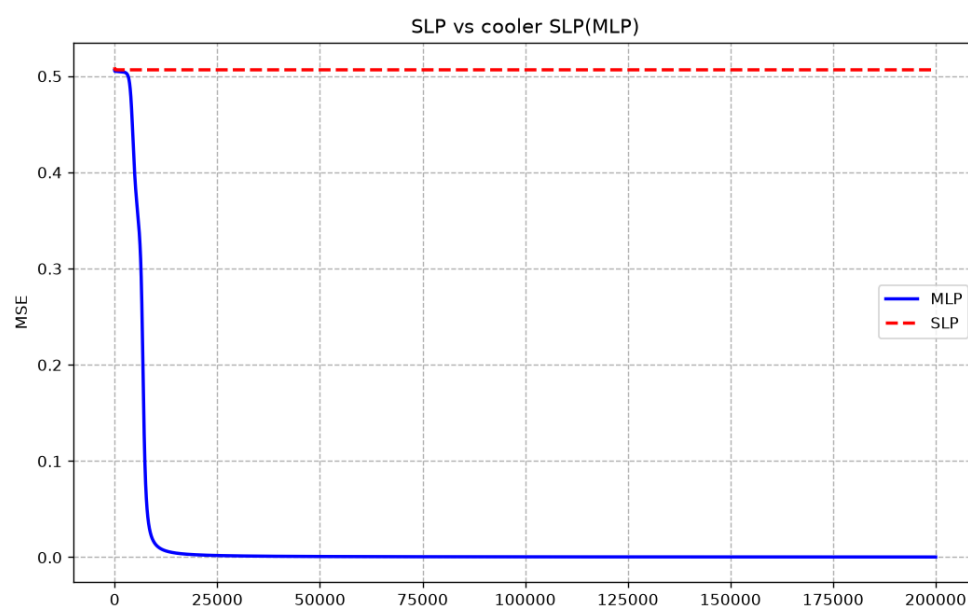
        a = float(parts[0])
        b = float(parts[1])

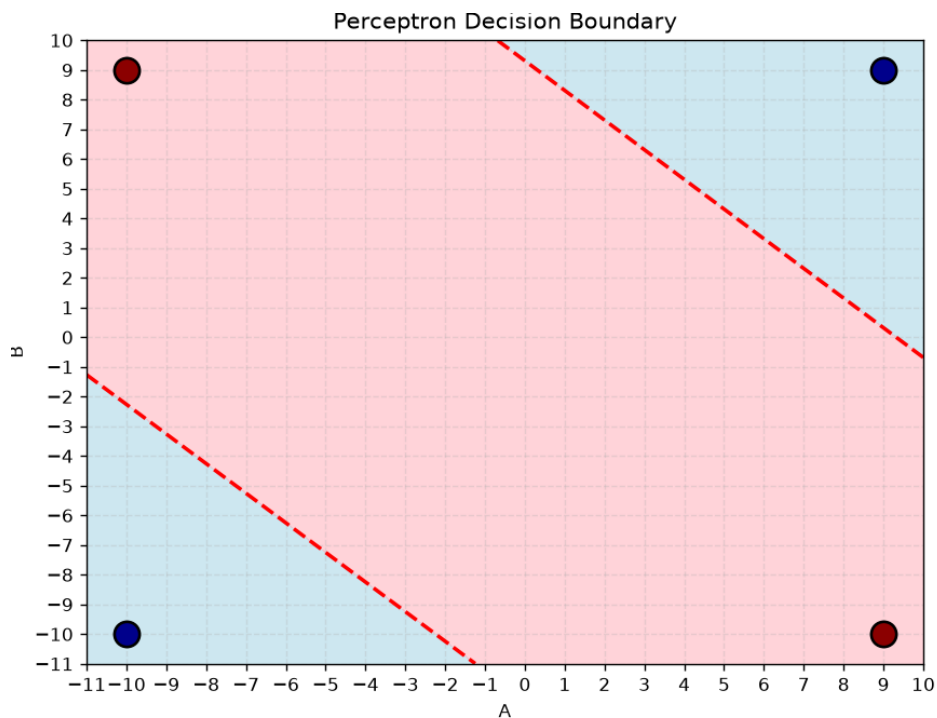
        print(f"--- Point A={a}, B={b} ---")
        print("MLP (2-2-1):")
        predict_point(net_mlp, a, b, c0, c1)

    except ValueError:
        print("Error: invalid input!")

```

Результат





```

Input: A=-10, B=-10 Output: -9.86 Predicted class: -10
Input: A=-10, B=9 Output: 8.84 Predicted class: 9
Input: A=9, B=-10 Output: 8.84 Predicted class: 9
Input: A=9, B=9 Output: -9.85 Predicted class: -10
Input: A=0, B=0 Output: 8.82 Predicted class: 9
Input: A=5, B=-5 Output: 8.82 Predicted class: 9

Input nubers A and B through the space(or 'q' for exit): -3 -10
--- Point A=-3.0, B=-10.0 ---
MLP (2-2-1):
Input: A=-3.0, B=-10.0 Output: -3.85 Predicted class: -10

Input nubers A and B through the space(or 'q' for exit): 0 -10
--- Point A=0.0, B=-10.0 ---
MLP (2-2-1):
Input: A=0.0, B=-10.0 Output: 6.93 Predicted class: 9

Input nubers A and B through the space(or 'q' for exit): 

```

Вывод, ответы на вопросы

1. Однослойный персептрон в данном эксперименте не смог корректно решить задачу XOR: после обучения точность осталась на уровне 50%. Причина заключается в том, что классы XOR являются линейно неразделимыми, поэтому одной линейной границы для их разделения недостаточно.

2. Многослойный персептрон с архитектурой 2-2-1 успешно решил задачу XOR и достиг точности 100%. По результатам обучения правильная классификация

всех примеров была достигнута примерно на 8000-й эпохе, после чего значение ошибки продолжало постепенно уменьшаться.

3. Использование произвольных числовых классов вместо 0 и 1 изменило масштаб целевых и выходных значений, но не изменило принцип решения задачи. Поэтому для выходного слоя необходимо выбирать функцию активации, которая может работать с требуемым диапазоном значений, включая отрицательные числа.

4. Хотя бы один скрытый слой необходим для решения XOR, так как он позволяет выполнить нелинейное преобразование входных данных. После такого преобразования классы становятся разделимыми на выходном слое, что невозможно реализовать обычной линейной моделью.

5. Использование классов из диапазона $[-10;10]$ принципиально не изменило сложность задачи по сравнению с классическим XOR, где используются 0 и 1. Структура расположения классов и их линейная неразделимость остались прежними, изменился только масштаб числовых значений, используемых при обучении.